Atmospheric Modeling with WRF: A Practical Introduction

L. Fernando Pareja-Roman

Numerical Modeling of the Ocean and Atmosphere I (16:712:603)

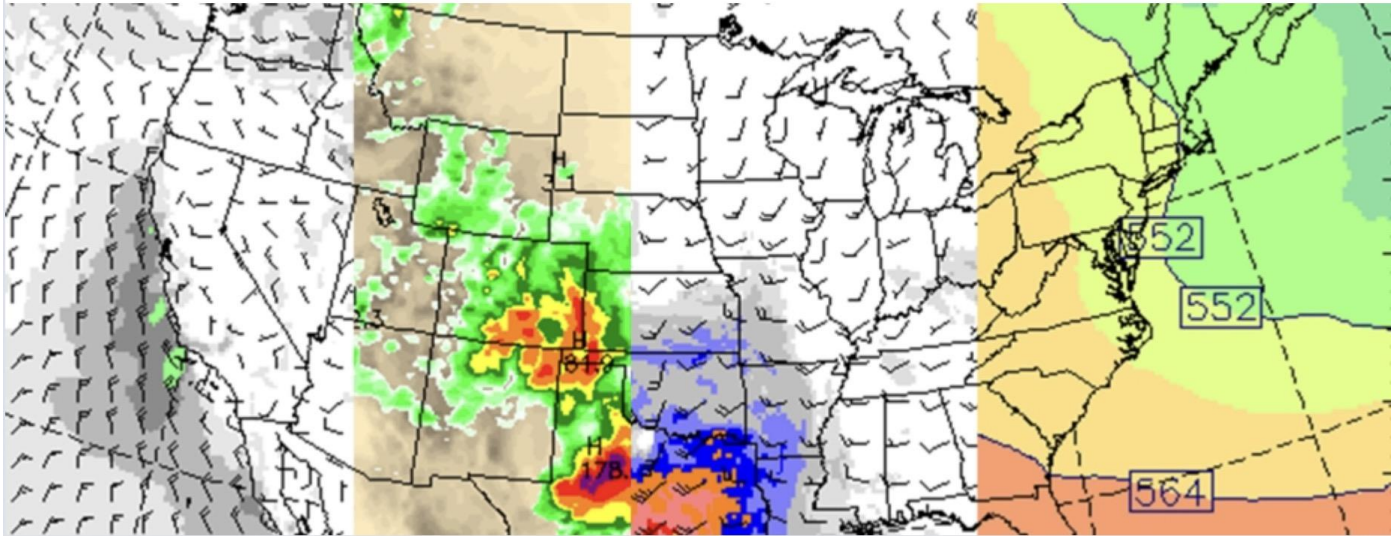
October 10th 2024

Weather Research and Forecasting (WRF) model

A state-of-the-art mesoscale numerical weather prediction system designed for both atmospheric research and operational forecasting applications

Main site: https://www.mmm.ucar.edu/models/wrf
User site: https://www2.mmm.ucar.edu/wrf/users/

Current Version 4.5



Goals for today

- **Not a full WRF tutorial!** See https://www2.mmm.ucar.edu/wrf/OnLineTutorial/
- Manual: https://www2.mmm.ucar.edu/wrf/users/docs/user_guide_V3/user_guide_V3.9/ARWUsersGuideV3.9.pdf
- General overview on WRF
- Based on the Rutgers Amarel system

- Test-case based for class. Become familiar with:
  - Basic requirements for WRF
  - Recipes (sequence of steps critical for WRF)
  - Making your own geographic domains
  - Experiment with physical parameterizations, time steps, spatial resolution and other options
  - Confidence building! Baseline for future projects

Later we'll see

Planetary Boundary Layer Physics

Surface Layer Physics (Air-Sea Interaction)

What you need from the Web

1. **WRF** source code

https://github.com/wrf-model/WRF.git

2. **WPS** source code (WRF Pre-processing system)

https://github.com/wrf-model/WPS.git

3. **WPS_GEOG** database (WRF Pre-processing system)

Website: https://www2.mmm.ucar.edu/wrf/users/download/get_sources_wps_geog.html
Actual link to database: https://www2.mmm.ucar.edu/wrf/src/wps_files/geog_high_res_mandatory.tar.gz
This database is 29 GB in size

Ideally you would have three separate folders and an additional one where your project or case studies will be:

/your/path/**WPS**
/your/path/**WRF**
/your/path/**WPS_GEOG**
/your/path/**nMOA_test**

The WRF/WPS Catch 22

The ultimate goal is to run the WRF model.

- To run a realistic case of WRF, you need output from the pre-processing system (WPS)
- To run WPS, you need a compiled WRF code

Once you have these folders below:

/your/path/**WPS**
/your/path/**WRF**
/your/path/**WPS_GEOG**
/your/path/**nMOA_test**

Your next steps would be:

1. **Build WRF first**
2. **Build WPS second (because you will need to give the path to compiled WRF)**
3. **WPS_GEOG is a geographic dataset and doesn't need to be built**

"Building" (same as "compiling") results in the creation of critical (.exe) files needed to run both WPS and WRF

Building WRF can be time-consuming (1+ hour)!

I will provide compiled WRF and WPS source codes today
See handout later if you want to build your own WRF and WPS

Once you have compiled WRF and WPS

Your folders now will be: (**red means compiled**)

/your/path/**WPS**
/your/path/**WRF**
/your/path/**WPS_GEOG**
/your/path/**nMOA_test**

When you build WRF from the nMOA_test folder, it should look like this (with numerous symbolic links to the compiled WRF source code).

You should see a link to wrf.exe

```
(mercator) [lfp33@amarel1 build_example]$ ls
aerosol.formatted                    CLM_ASM_ICE_DRC_DATA                ndown.exe
aerosol_lat.formatted                CLM_DRDSDT0_DATA                    ozone.formatted
aerosol_lon.formatted                CLM_EXT_ICE_DFS_DATA                ozone_lat.formatted
aerosol_plev.formatted               CLM_EXT_ICE_DRC_DATA                ozone_plev.formatted
BROADBAND_CLOUD_GODDARD.bin          CLM_KAPPA_DATA                      p3_lookupTable_1.dat-v5.4_2momI
Build_wrf                            CLM_TAU_DATA                        p3_lookupTable_1.dat-v5.4_3momI
build_wrf.sh                         co2_trans                           p3_lookupTable_2.dat-v5.3
bulkdens.asc_s_0_03_0_9              coeff_p.asc                         real.exe
bulkradii.asc_s_0_03_0_9             coeff_q.asc                         RRTM_DATA
CAM_ABS_DATA                         constants.asc                       RRTMG_LW_DATA
CAM_AEROPT_DATA                      create_p3_lookupTable_1.f90-v5.4    RRTMG_SW_DATA
CAMtr_volume_mixing_ratio            create_p3_lookupTable_2.f90-v5.3    SOILPARM.TBL
CAMtr_volume_mixing_ratio.A1B        eclipse_besselian_elements.dat      SOILPARM.TBL_Kishne_2017
CAMtr_volume_mixing_ratio.A2         ETAMPNEW_DATA                       STOCHPERT.TBL
CAMtr_volume_mixing_ratio.RCP4.5     ETAMPNEW_DATA.expanded_rain         tc.exe
CAMtr_volume_mixing_ratio.RCP6       GENPARM.TBL                         termvels.asc
CAMtr_volume_mixing_ratio.RCP8.5     grib2map.tbl                        tr49t67
CAMtr_volume_mixing_ratio.SSP119     gribmap.txt                         tr49t85
CAMtr_volume_mixing_ratio.SSP126     HLC.TBL                             tr67t85
CAMtr_volume_mixing_ratio.SSP245     ishmael-gamma-tab.bin               URBPARM_LCZ.TBL
CAMtr_volume_mixing_ratio.SSP370     ishmael-qi-qc.bin                   URBPARM.TBL
CAMtr_volume_mixing_ratio.SSP585     ishmael-qi-qr.bin                   URBPARM_UZE.TBL
capacity.asc                         kernels.asc_s_0_03_0_9              VEGPARM.TBL
CCN_ACTIVATE.BIN                     kernels_z.asc                       wind-turbine-1.tbl
CLM_ALB_ICE_DFS_DATA                 LANDUSE.TBL                         wrf.exe
CLM_ALB_ICE_DRC_DATA                 masses.asc
CLM_ASM_ICE_DFS_DATA                 MPTABLE.TBL
```

What you need to run WPS and WRF

WRF Requirements:

WRF Input file: **namelist.input**
Initial condition file: **wrfinp_d01**
Lateral boundary conditions: **wrfbdy_d01**

To obtain **wrfinp_d01** and **wrfbdy_01**, you need to run all the components of WPS (geogrid.exe, ungrib.exe, metgrid.exe) and real.exe. This needs a separate **namelist.wps.** I will provide templates and code that do this automatically.

How to get wrfinp_d01 and **wrfbdy_01 with WPS**

Order	Component	Function	Needs	Output
1	geogrid.exe	Obtains geographic dataset (land use, continents, topography, etc)	namelist.wps WPS_GEOG GEOGRID.TBL	geo_em_d01.nc
2	ungrib.exe	Gets global model output in GRIB format and converts them to and intermediate file format	namelist.wps .grib2 files from GFS, ERA, NAM, etc link_grib.sh Vtable	GFS:YYYY-MM-DD_HH files
3	metgrid.exe	Combines geographic dataset and ungribbed data to create meteorological gridded data	namelist.wps, GFS:YYYY-MM-DD_HH files and geo_em_d01.nc METGRID.TBL	met_em.d01_....nc files
4	real.exe	Creates WRF initial and boundary conditions	namelist.wps, met_em.d01_....nc files and namelist.input	**wrfinp_d01** **wrfbdy_01**

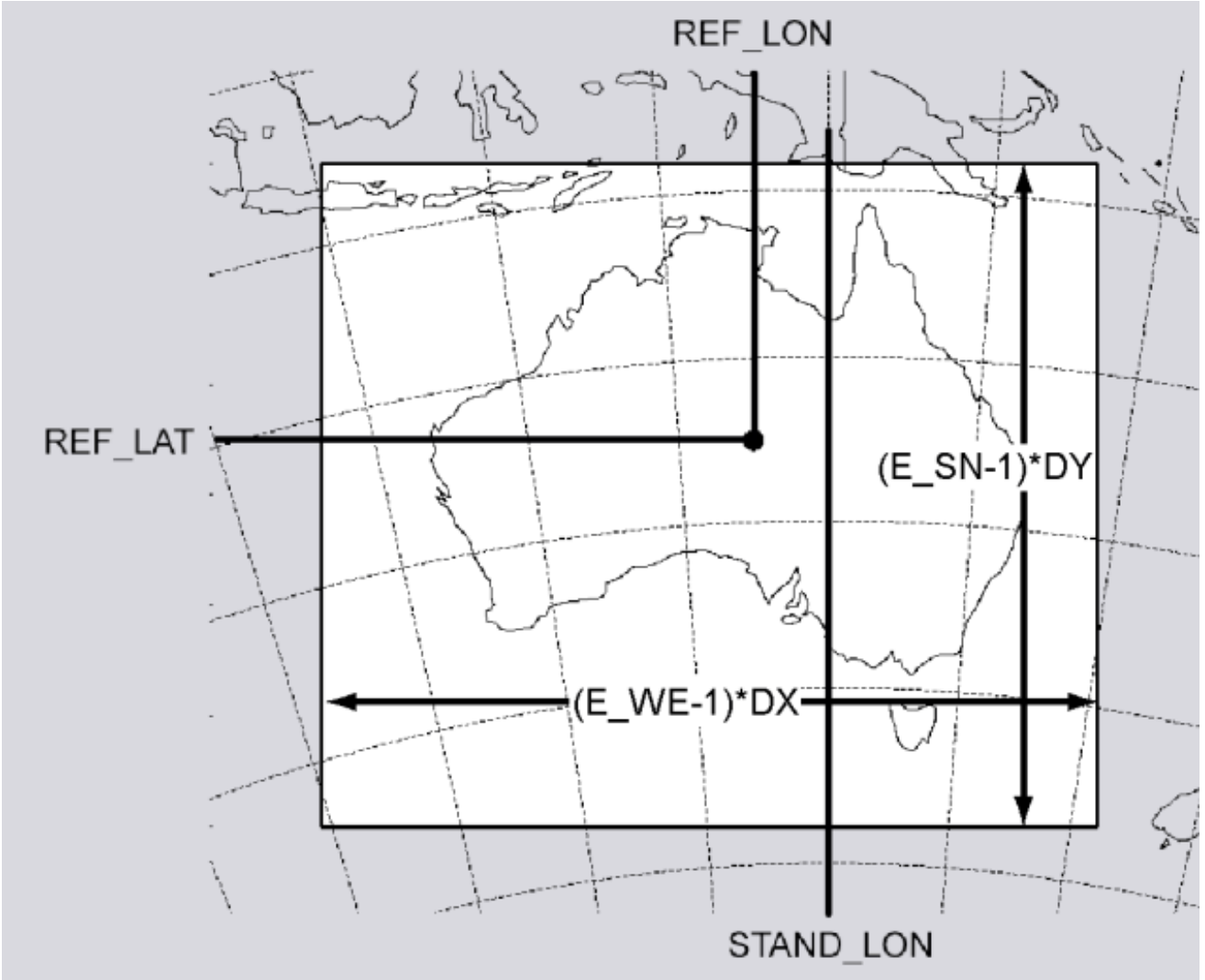
Creating your own geographic domains: must update both **namelist.input** and **namelist.wps**

Example Geogrid section

```
&geogrid
 parent_id                 = 1
 parent_grid_ratio         = 1
 i_parent_start            = 1
 j_parent_start            = 1
 e_we                      = 320
 e_sn                      = 280
 geog_data_res             = '30s'
 dx                        = 6000
 dy                        = 6000
 map_proj                  = 'mercator'
 ref_lat                   = 10.49
 ref_lon                   = -75.48
 truelat1                  = 10.49
! truelat2                   = 24.47
! stand_lon                  = -87.23
 ref_x                     = 160
 ref_y                     = 140
```

Mercator for low latitudes
Lambert for mid-latitudes

Graphical representation of geogrid parameters



Setting up your own WRF Test Run

There is a folder on Amarel for everything related to the atmospheric and air-sea interaction

/projects/dmcs_1/courses/nmoa/weather/

Similar to the folder structure we discussed, we have these folders in there:

WPS (compiled)
WRF (compiled)
WPS_GEOG
nMOA_test

The **nMOA_test** folder is a directory you can **copy** to your project or personal folder to work on your own test cases. Please do not run tests in this folder!

You don't need to copy WPS, WRF or WPS_GEOG.
The scripts you will use later point to these folders

Once you have a copy of the **nMOA_test** folder, these are contents:

File or folder name	Description
build_wrf.sh	This is the script to build WRF and copy relevant links to the current directory. Since I have already compiled WRF, you don't need to run this
Build_wrf/	A directory that contains copies of the compiled .f90 WRF code. This is for reference. Good to check what was actually built. At this stage, we don't need to worry about this.
CAMtr*, CCN*, ETAM*, etc..	These are symbolic links to files in the WRF source code. They need to be in the folder where WRF will be eventually run, so we're stuck with this clutter for now. Do not remove.
iofields_list.txt	This file specifies what variables are requested in the main history file (WRF output). Example: **+:h:0:UOCE** (adds the variable UOCE to the output and so on), while **-:h:0:TISO** removes the variable TISO from the output). No spaces here.
wrf.exe	This is the executable to run WRF. It's a link from the WRF source code.
namelist.wps.tmp	**User input required:** Template to create **namelist.wps**, which is needed to run WPS
namelist.input.tmp	**User input required:** Template to create **namelist.input**, which is needed to run WPS
wps_user_input.sh	**User input required:** script used to select start date and number of run days to create namelists based on the templates
submit_wps.sh	Job script to submit wps_user_input.sh. Change e-mail here to your address: #SBATCH --mail-user=user@marine.rutgers.edu
submit_wrf.sh	Job script to run wrf.exe based on namelist.input Change e-mail here to your address: #SBATCH --mail-user=user@marine.rutgers.edu

Example case: Winds over the Kuroshio

This is the general workflow to create the model we worked on in class.

1. Pick the projection: **mercator** (low latitudes), **lambert** (mid-latitudes)
2. Pick the geographic parameters below for your model. Edit **namelist.wps.tmp** accordingly in the &geogrid section:

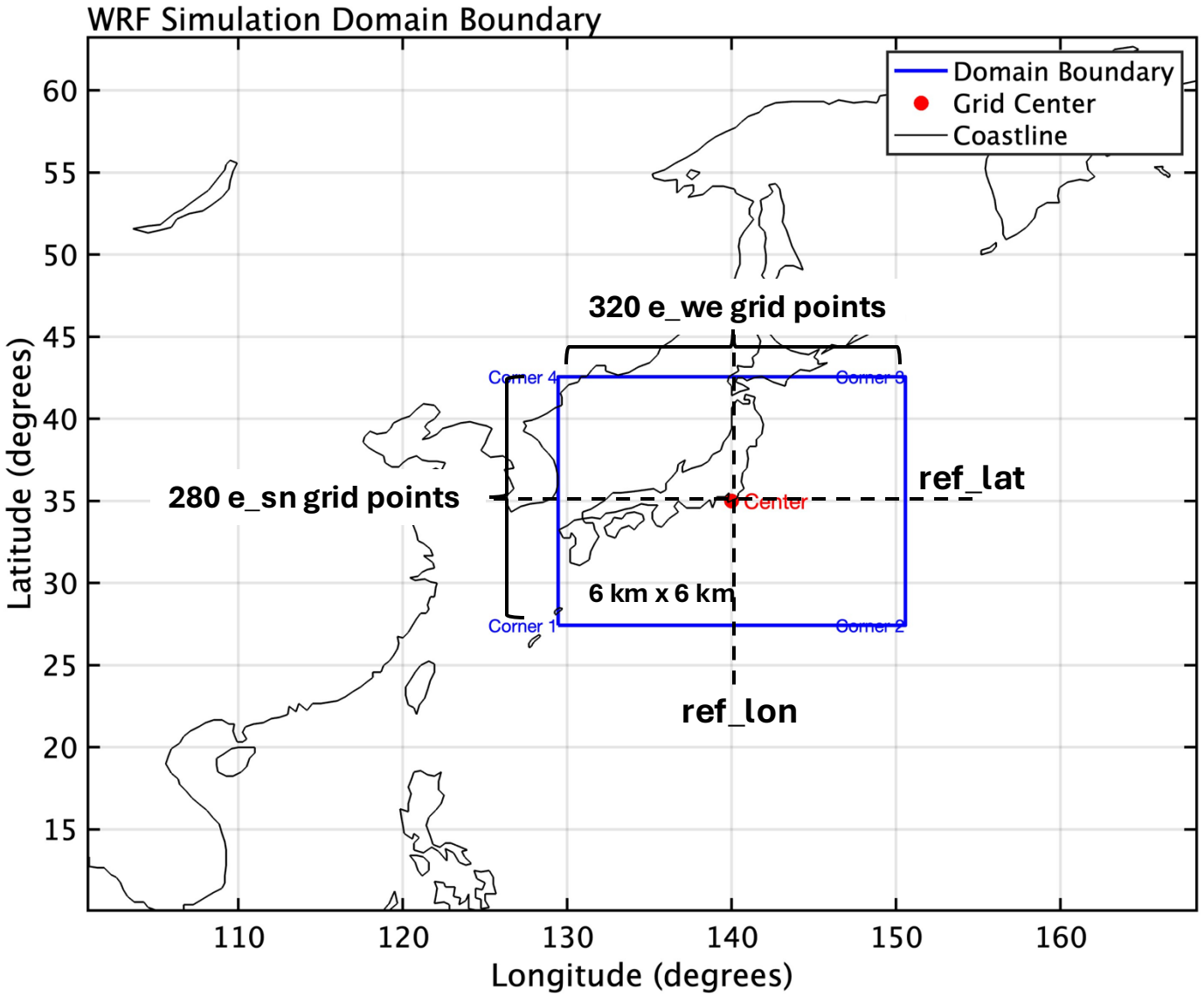
```
e_we      = 320 ! Number of grid points west to east
e_sn      = 280 ! Number of grid points east to west
dx        = 6000 ! x-grid spacing in meters
dy        = 6000 ! y-grid spacing in meters
map_proj  = 'lambert' ! choose from Mercator (low latitude) or Lambert (mid-latitude)
ref_lat   = 35.0 ! Lat of Japan (center of domain)
ref_lon   = 140.0 ! Lon of Japan (center of domain)
truelat1  = 35.0 ! set for both Mercator and Lambert. Try = ref_lat
truelat2  = 35.0 ! set for Lambert only. Try = ref_lat
stand_lon = 140.0 ! set for Lambert only. Try = ref_lon
ref_x     = 160 (half of e_we)
ref_y     = 140 (half of e_sn)
```

3. Edit **namelist.input.tmp** with the same values in the &domain section

```
e_we  = 320,
e_sn  = 280,
dx    = 6000.0,
dy    = 6000.0,
```

This is also where you pick the time_step of your model (in this example set to 20 seconds) as well as the &physics and &dynamics options. For a full explanation of namelist.input parameters, see: https://pubs.usgs.gov/sir/2014/5089/downloads/namelist.README

Optional: see **plot_wps_domain.m** MATLAB script to get an estimate of what the domain will look like (approximately). For the Kuroshio, we picked:



Once you have edited **namelist.wps.tmp** and **namelist.input.tmp**, edit **wps_user_input.sh.**
Here, enter the **START_DATE** in the format shown and the number of **RUN_DAYS**.

INTERVAL_SECONDS is the time interval to import lateral boundary conditions from GFS.
For now, leave as-is (21600 seconds = 6 hours)

```
# ----------------------------
# Set start date and run days (no spaces). This will be used by WPS components and REAL
# ----------------------------

# Start Date in "YYYY-MM-DD_HH:MM:SS" format
START_DATE="2024-01-15_00:00:00"
RUN_DAYS=1

# Define INTERVAL_SECONDS below (Requested frequency of incoming open boundary conditions)
# 3 hours = 10800 seconds
# 6 hours = 21600 seconds

INTERVAL_SECONDS=21600
```

With these complete:

- **namelist.wps.tmp**
- **namelist.input.tmp**
- **wps_get_input.sh**

You may submit:

```
sbatch submit_wps.sh
```

The script above will create input files following the templates and instructions. It will then run geogrid.exe, ungrib.exe, metgrid.exe, and real.exe). The final result will be:

- **namelist.input** (namelist for WRF)
- **namelist.wps** (namelist for WPS)
- **wrfinp_d01** (Initial condition file for WRF)
- **wrfbdy_d01** (Lateral boubdary condition file for WRF)
- **logs/** folder (folder with logs for geogrid, ungrib, metgrid, and real)
- **log.wps** (log file for the whole process)

The last step is to run WRF:

```
sbatch submit_wrf.sh
```

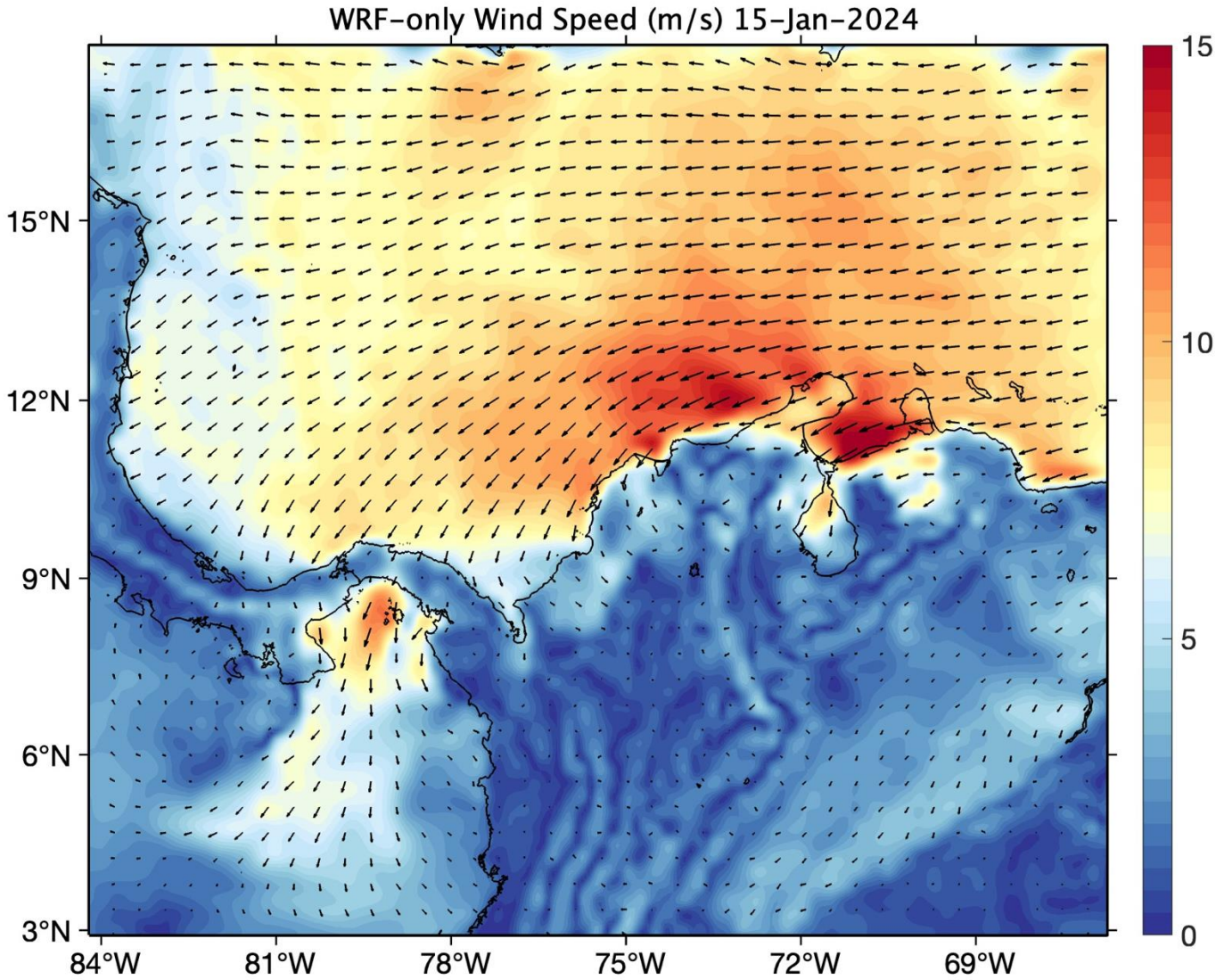
When WRF starts running, the log is rsl.error.XXXX.

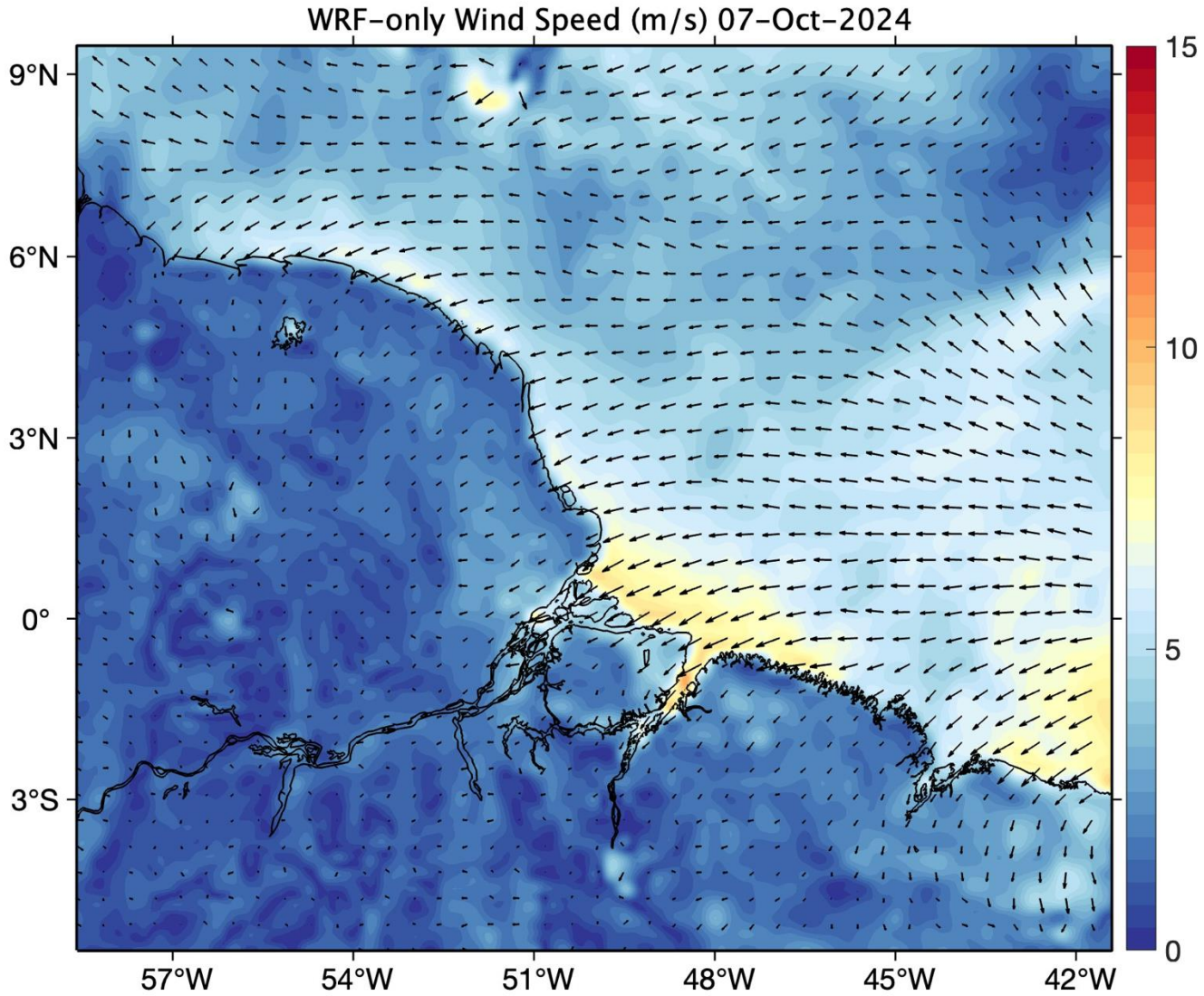
Issue `tail -f rsl.error.0000` to see if it's running. When the run is complete, the log is merged into **log.wrf**.

The output file will be called: **wrf_his_d01.nc**

Other examples: Caribbean Low Level Jet (CLLJ), Trade Winds. Jan 2024

e_we = 320
e_sn = 280
geog_data_res = '30s'
dx = 6000
dy = 6000
map_proj = 'mercator'
ref_lat = 10.49
ref_lon = -75.48
truelat1 = 10.49
ref_x = 160
ref_y = 140



Other examples: Onshore wind in Amazon Delta. Oct 2024

e_we = 320
e_sn = 280
geog_data_res = '30s'
dx = 6000
dy = 6000
map_proj = 'mercator'
ref_lat = 4.0
ref_lon = -50.0
truelat1 = 4.0
ref_x = 160
ref_y = 140